

Kuromaku

A versioned marketing memory with provenance, staleness propagation, and a planner that reports what it cannot do.

Built as a rebuild of Okara, an "AI CMO" product, with the architectural changes described in section 2. The seed workspace is ShogunAI, a pre-launch macOS app that captures work context into a private local memory.

Live instance: <https://kuromaku-nine.vercel.app>

Repository: <https://github.com/gsaianimesh/kuromaku>

Engineering documentation: docs/ in the repository, sixteen files

Demo walkthrough: section 4 of this document

0. How to read this

This document is the demo. There is no video, so section 4 carries the walkthrough: every screen, in the order you would visit it, narrated closely enough that you need not open the application.

The rest is context. Section 1, what the system is. Section 2, the defects it was built against. Section 3, how it fits together. Section 5, verification. Section 6, what is missing.

About the screenshots

Every image in this document was captured from the running application against its real database, by a committed script.

```
scripts/screenshots.ts    twenty static shots
scripts/pairs.ts          before and after pairs for state changes
```

Each navigates to a real route, waits for a selector proving the content rendered, and **refuses to capture** if it did not: a missing state produces a named error, never a placeholder. The screenshot script also scans the settings page for anything shaped like an API key and aborts rather than write an image with a credential in it. Nothing was mocked, staged, or edited after capture. Regenerate with `npm run dev`, then `npm run screenshots` and `npm run pairs`.

A static document cannot show a transition, so the five state changes that matter appear twice, labelled *before* and *after*. Each change was made by calling the real code path, never by writing the after state into the database.

What you can check yourself

The live instance is a demo workspace holding no credentials and no private data, deliberately left open so the claims below can be checked; the actions that would spend the owner's model credits are refused there, and `/health` reports whether that gate is on. Section 6 explains why an authenticated deployment would need more than this.

At the live URL, without any setup:

- `/memory` shows the compiled records with their sources, their derivation parents and their confidence. Compare any record to the same one in this document.

- `/planner` shows which prioritised channels have no agent.
- `/api/v1/memory` returns the same records as JSON, each carrying its sources, a `grounding` value and the records it was compiled from.
- `/api/mcp` returns the MCP tool manifest.
- `/health` reports environment, database, migration and encryption checks, and answers 503 if any fails.

In the repository:

- `docs/` explains every claim in this document at implementation depth, with file paths.
- `npm run verify` runs the infrastructure checks with no model calls.
- `npm run e2e` walks the whole pipeline against real model calls.
- `npm run eval` runs the golden set.

1. What this is

Kuromaku crawls a company website, compiles the text into versioned records called a memory, and runs channel agents that draft from it. Drafts go to a human. Published work accumulates observations, and those observations change what gets scheduled next.

One idea from the product it rebuilds is worth keeping: the shared strategy layer. Agents do not each re-derive context, they read one compiled set of records, and that is why output stays consistent across channels. Everything else here exists to fix something that layer does not do.

Five commitments

Each is visible in the interface, not only in the code.

Provenance on every fact. Every record accounts for itself in one of three ways: a source URL, a human assertion, or a named set of records it was compiled from. All three carry a confidence value, and the third cannot claim more confidence than the least certain record beneath it. A record with none of the three is still written, flagged in red, and capped at 0.4.

Versioned memory with staleness propagation. Records are append only. Editing supersedes rather than overwrites. Every artifact records which memory records it came from, and editing a record marks derived artifacts stale, transitively.

The agent set is derived from the strategy. The planner reads compiled channel priorities and compares them against a code-seeded agent registry. A prioritised channel with no agent becomes a visible coverage gap, not silence.

Evidence, not justification. Every draft ships with the memory record ids, source links and data points it used. Each is clickable and resolves to something real.

Performance closes the loop. Observations feed the planner. A channel whose recent drafts have no observations stops being scheduled, and the planner says why.

What it explicitly does not do

These are enforced, not merely unbuilt.

It does not publish anything. No code path makes an authenticated write to any platform. `markAsPosted` records a URL a human supplies, and rejects any artifact that is not already approved.

It does not display an unmeasured number. Where nothing was observed the interface says so in words. A model with no entry in the pricing table produces a null cost that renders as "unpriced", never as \$0.00.

It does not authenticate users. Version one is single tenant with no login. Every table carries a workspace id, so multi tenancy is a routing change rather than a migration, but no such routing exists. The public demo compensates with a refusal list, not a login: `DEMO_MODE=1` blocks compiling, crawling, draining the queue and writing a model key. That is a stopgap for one deployment, not a security model, and section 6 says so.

2. What it was built against

These defects come from a first hand session with Okara, not from its marketing. Each one became a requirement. Three are shown here in Okara's own interface; the full session record is the companion document `Okara-Recon-Notes.pdf`.

Strategy and execution were disconnected. Its strategy ranked Product Hunt third and founder communities fourth. It had no agent for either. It did ship a UGC video agent and an influencer agent, which the strategy never asks for. The catalogue was fixed, and nothing in the product noticed the mismatch.

This is the defect the planner exists to answer: it compares compiled priorities against the agent registry and writes a coverage gap wherever nothing can execute. Step 3.

The thirty day roadmap was a dead document. Empty checkboxes in a PDF. Here, roadmap items compile into memory records and the planner turns them into jobs with real status.

No staleness tracking. Editing the ICP left every draft derived from it unchanged and unflagged. Records here are append only, and editing one marks every artifact downstream of it stale through the derivation chain. Step 2, which is the step worth reading if you only read one.

Rationale without evidence. Drafts carried a prose "why this works" panel — no links, no named threads, no data, no way to tell which part of the strategy informed the choice. Replaced with structured evidence on every draft, each item resolving to a memory record page or a real URL. Step 4.

Fabricated metrics. Unpublished drafts rendered invented like and view counts.

Nothing in this system displays a number that was not observed, and empty states say why they are empty. Step 5.

Duplicate execution. The onboarding pipeline ran twice in one session, including every research search, and one query fired twice inside a single run. Every job here carries an idempotency key enforced by a partial unique index; research is cached and deduplicated by normalised query hash. That normalisation had a bug of its own, which section 5 describes.

No provenance. Facts were asserted flat. It listed integrations and a latency claim with no source, and once a wrong fact was in memory every agent repeated it. This is the one the whole design is bent around. Citations are resolved against what the model was actually shown, so an index it invents resolves to nothing and is dropped. What is left then decides the record's ceiling: a record with a resolved citation keeps the confidence the model gave it, a record with no citation but with parents in the derivation graph cannot exceed the least confident record it rests on, and a record with neither is capped at 0.4 and flagged in red. Step 1.

Voice inferred from a landing page. Socials were skipped and it carried on, taking brand voice from marketing copy. Voice rules here compile per locale, and a locale with no source material in that language yields low confidence rules that state the gap rather than translated rules presented as observed.

Performance was optional and unused. The pipeline completed with Search Console and Analytics skipped and nothing degraded. Observations are a planner input here, so their absence changes what gets scheduled. Step 5.

3. How it fits together

Next.js on the App Router, TypeScript, Tailwind, Neon Postgres through Drizzle, Zod for validation. Model calls go to Groq, with Anthropic behind the same interface. Jobs run on a Postgres queue rather than a third party service.

The pipeline

```
crawl  —> sources      pages, deduplicated by content hash
      |
compile —> memory_records + record_sources + record_derivations
      |
      |           nine record types, provenance, derivation graph
plan   —> jobs         with a plain language reason
      |
      |           or coverage_gaps where no agent exists
run    —> artifacts + artifact_evidence
      |
      |           drafted, critiqued, evidence attached
review —> reviews     approve, edit with distance, or reject
      |
publish —> external_url a human posts it and confirms
      |
observe —> observations measured figures, never generated
      |
      └─> back into plan
```

Where the interesting decisions are

Claiming a job is one statement, not a transaction. The Neon HTTP driver runs each statement in its own implicit transaction, so a standalone `SELECT ... FOR UPDATE SKIP LOCKED` would drop its lock before the follow up update and two workers could claim the same row. The select is nested inside the update:

```
UPDATE jobs
SET status = 'running', locked_at = now(), attempts = attempts + 1
WHERE id = (
  SELECT id FROM jobs
  WHERE status = 'queued' AND run_after <= now()
  ORDER BY run_after ASC, created_at ASC
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
RETURNING *;
```

The idempotency index is partial. It applies only while a job is queued or running:

```
CREATE UNIQUE INDEX "jobs_idempotency_key_uq"
ON "jobs" USING btree ("idempotency_key")
WHERE status in ('queued', 'running');
```

The queue's job is preventing concurrent duplicate execution. Terminal rows release their key because re-running is required behaviour: re-compiling must supersede rather than duplicate, and re-crawling must be possible. Not scheduling redundant work is a planning decision and lives in the planner, which has the history to judge it.

Provenance is enforced after the model returns, never trusted from it. Each stage gives the model a numbered source list. The model returns indices. Those indices are then resolved against the list it was actually shown:

```
for (const idx of record.sourceIndices) {
  const source = sourceRows[idx];
  if (source) {
    citations.push({ sourceId: source.id, url: source.url, snippet: record.snippet });
  }
}
```

An index out of range resolves to `undefined` and is dropped. A URL the model invented is not found in the search results and is dropped. What is not dropped is the record itself.

Only three of nine compile stages receive raw page text. Product facts, ICP segments and voice rules read the company's own prose. The rest reason from records already compiled, which is the shared strategy layer doing its job, and also what keeps each prompt inside the provider's per minute token budget.

4. Guided walkthrough

Five steps, one per commitment, in the order you would actually visit them.

Step 1. Provenance on every fact

Start at `/memory`. This is the compiled memory: nine record types, each row carrying a value, a confidence score, a locale, an origin and a version.

kuzomaku v0

OverviewSourcesMemoryReviewPlannerPublishMetricsJobsSettingsHealth

46 active31 sourced15 derivedavg confidence 0.74enja

COMPILER

8 source(s) available

Queue compile

Run compile now

Re-compiling supersedes records rather than duplicating them.

PRODUCT FACTS

8 records

contextual-reply-drafting0.90encompiledv9history

{
 "fact": "The AI can draft contextual replies, e.g., follow-up messages.",
 "category": "capability"
}

source: [ShogunAI — Memory that acts](#)
"Drafted follow-up to Mika"

edit

hybrid-search0.95enhumanv11history

{
 "fact": "ShogunAI provides hybrid search across all captured content.",
 "note": "Corrected during review.",
 "category": "capability",
 "correctedByHuman": true
}

source: Asserted by a human via the memory editor.

edit

macos0.90encompiledv9history

{
 "fact": "ShogunAI runs on macOS.",
 "category": "platform"
}

source: [ShogunAI — Memory that acts](#)
"No credit card : macOS : Private by default"

edit

no-credit-card0.90encompiledv9history

{
 "fact": "No credit card is required to join the early-access waitlist.",
 "category": "pricing"
}

The header counts active records, how many are sourced, derived and ungrounded, the average confidence, and the locales present. Each type panel repeats its own counts in the hint, so the quality of the memory is legible before reading a single record.

Open any product fact and look underneath the value.

contextual-reply-drafting0.90encompiledv9history

{
 "fact": "The AI can draft contextual replies, e.g., follow-up messages.",
 "category": "capability"
}

source: [ShogunAI — Memory that acts](#)
"Drafted follow-up to Mika"

edit

The badge row carries confidence, locale, origin and version. Below the JSON value are the resolved sources, each a link to the page it came from, and beneath each one, in quotes, the snippet the model cited. That snippet is text the model claimed appears in that source.

Not every record can cite a page. A positioning statement is not written anywhere on the site; the compiler builds it from product facts and ICP segments. Those records name what they were built from.

private-local-memory-vs-cloud-notes 0.50 en compiled v6 derived history

```
{
  "category": "privacy",
  "statement": "ShogunAI stores memory locally on the device and keeps data private by default, unlike cloud-based note services that sync everything to servers.",
  "differentiator": "on-device storage and default privacy",
  "againstAlternative": "cloud note-taking apps"
}
```

compiled from: product_fact: contextual-reply-drafting, product_fact: hybrid-search (superseded), product_fact: macos, product_fact: no-credit-card, product_fact: on-device-memory, product_fact: private-by-default, product_fact: quick-recall, product_fact: tool-integration, icp_segment: fast-movers (superseded), icp_segment: founders, icp_segment: macos-users
edit

No source line, because no page states this. Instead, `compiled from:` and the records it was built on, each a link. Following one reaches a record that does cite a page.

Confidence follows the same graph. A record with a citation keeps what the model gave it. A derived record is capped at the confidence of the least certain record beneath it — it cannot be surer than its own foundation, and the number comes from the graph rather than a constant someone picked:

```
const parentFloor =
  parents.length > 0 ? Math.min(...parents.map((p) => p.confidence)) : null;

const confidence =
  citations.length > 0
    ? emitted.confidence
    : parentFloor !== null
      ? Math.min(emitted.confidence, parentFloor)
      : Math.min(emitted.confidence, 0.4);
```

The last branch is the one that earns a red warning: a record with no citation *and* no parent, which nothing in the system can account for. It is written anyway rather than dropped — dropping it would make the memory look cleaner than it is — then capped at 0.4 and flagged.

There are none of those in this workspace. Every one of the 46 active records is either sourced or derived. That is worth stating rather than quietly omitting, because an earlier version of this document showed a screenshot of the warning: back then derivation did not count as provenance, and fifteen records that the compiler could fully account for were being displayed in red at 0.40. Fixing that emptied the category. The path still exists, the golden set still asserts it, and a product fact the compiler cannot ground would still land there — that stage has no parents to fall back on.

The competitors section is worth a look for the opposite reason.

atlas-ai-store 0.90 en compiled history

```
{
  "url": "https://atlas.ai/store",
  "name": "Atlas: AI Store",
  "overlap": "Both provide AI components to enhance workflows.",
  "positioning": "AI component marketplace for developers and creators.",
  "differenceFromUs": "ShogunAI integrates AI directly into a personal memory layer, whereas Atlas offers modular AI services for integration into other products."
}
```

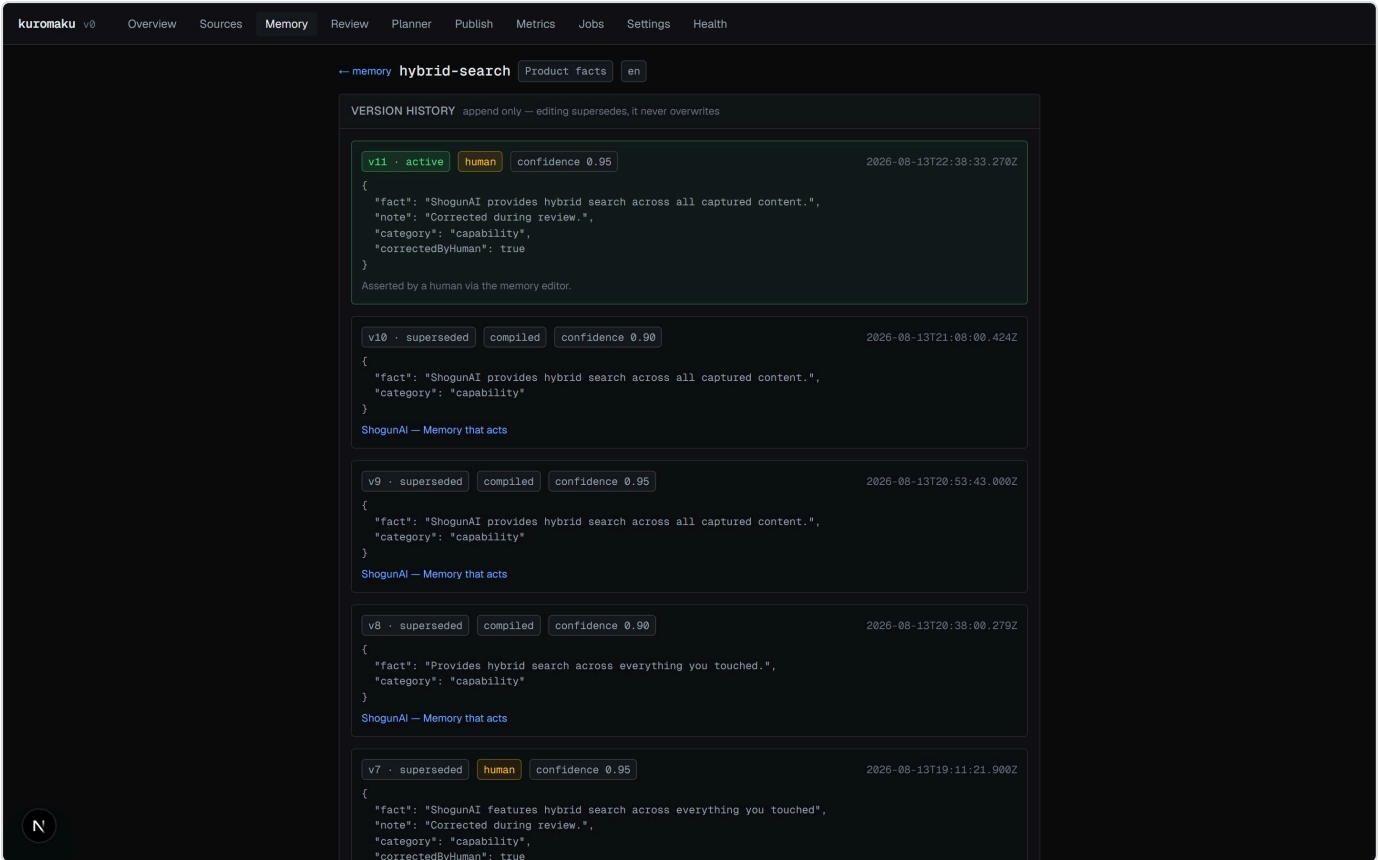
source: <https://letsmetrix.com/app/shogun-ai-section-builder/alternatives>
"Atlas: AI Store"
edit

The source URL here came from a web search, not the company's own site. Competitors are the one stage that runs research. With no search provider configured the section produces nothing at all rather than a list of plausible sounding names, and the compile log records why.

The defect this answers. Okara asserted facts flat, with no source. It listed integrations and a latency claim that were simply wrong, and every agent repeated them afterwards. The drafts in step 4 make a latency claim too — "results in about 0.2 seconds" — and that one traces to `product_fact: quick-recall` at 0.90 confidence, cited to the page it was read from. The difference is not that this system avoids specifics. It is that a specific has somewhere to lead.

Step 2. Versioned memory, and what an edit invalidates

This is the step that matters most. Records are append only: nothing is updated in place except the status column of the row being retired, and editing produces a new version that supersedes the old one. Open a record's history at `/memory/<id>` .



Every version, newest first, nothing deleted. The top row is active with origin `human` ; the rest are superseded and keep their own sources. Further down the same page a panel headed "What an edit here would invalidate" lists the records compiled from this one, grouped by how many hops away they are.

Here are the two ends of one chain, side by side.

VERSION HISTORY: THE SAME RECORD BEFORE AND AFTER A HUMAN CORRECTION

BEFORE

```
v10 · superseded  compiled  confidence 0.90  2026-08-13T21:08:00.424Z
{
  "fact": "ShogunAI provides hybrid search across all captured content.",
  "category": "capability"
}
ShogunAI — Memory that acts
```

superseded, origin compiled — what the compiler produced. Retained in full, with its own sources, readable forever.

AFTER

```
v11 · active  human  confidence 0.95  2026-08-13T22:38:33.270Z
{
  "fact": "ShogunAI provides hybrid search across all captured content.",
  "note": "Corrected during review.",
  "category": "capability",
  "correctedByHuman": true
}
Asserted by a human via the memory editor.
```

active, origin human, version advanced, value changed. The correction became a new row rather than an overwrite.

The human version still carries provenance. `editRecord` writes a source row recording the assertion:

```
await db.insert(recordSources).values({
  recordId: created.id,
  sourceId: null,
  url: null,
  snippet: "Asserted by a human via the memory editor.",
});
```

Without that row a human edited record would count as ungrounded and get a warning, which would be wrong. A person asserting something is a form of provenance, just not a URL.

What the edit does downstream

Here is a draft in the review queue immediately before a memory edit, and the same draft immediately after.

A DRAFT BEFORE AND AFTER A RECORD IT DEPENDS ON WAS CORRECTED

BEFORE

POST · PRODUCT_HUNT · Launch and community agent · en

critic 1.00 draft

Your work, remembered.

You get ShogunAI, a macOS-only assistant that captures activity from your daily tools and stores the memory locally. You can draft contextual replies—like follow-up messages—directly in the apps you use. It unifies everything with hybrid search, delivering fast, unified results. Because the memory lives on your device and is private by default, nothing is sent to a cloud server. You can join the early-access waitlist without a credit card.

What's different?

- On-device storage keeps your data private.
- Hybrid search across all captured content, not just files.
- Fast recall and contextual reply drafting built into your workflow.

EVIDENCE

contextual-reply-drafting · product_fact: contextual-reply-drafting
hybrid-search · product_fact: hybrid-search
macos · product_fact: macos
no-credit-card · product_fact: no-credit-card
on-device-memory · product_fact: on-device-memory
private-by-default · product_fact: private-by-default
quick-recall · product_fact: quick-recall
tool-integration · product_fact: tool-integration

CRITIC

Scored below threshold and was revised once automatically before reaching you.
No violations recorded.

PERFORMANCE

No observations recorded. Nothing is shown here that was not measured.

Approve

Edit and approve

Reject

job 056caf32 · created 2026-08-13T22:45:30.280Z

The evidence panel lists memory record keys as links and nothing is flagged. Ordinary pending work.

AFTER

Stale. The memory this draft rests on has changed since it was written:

- product_fact: no-credit-card was superseded

Regenerate from current memory

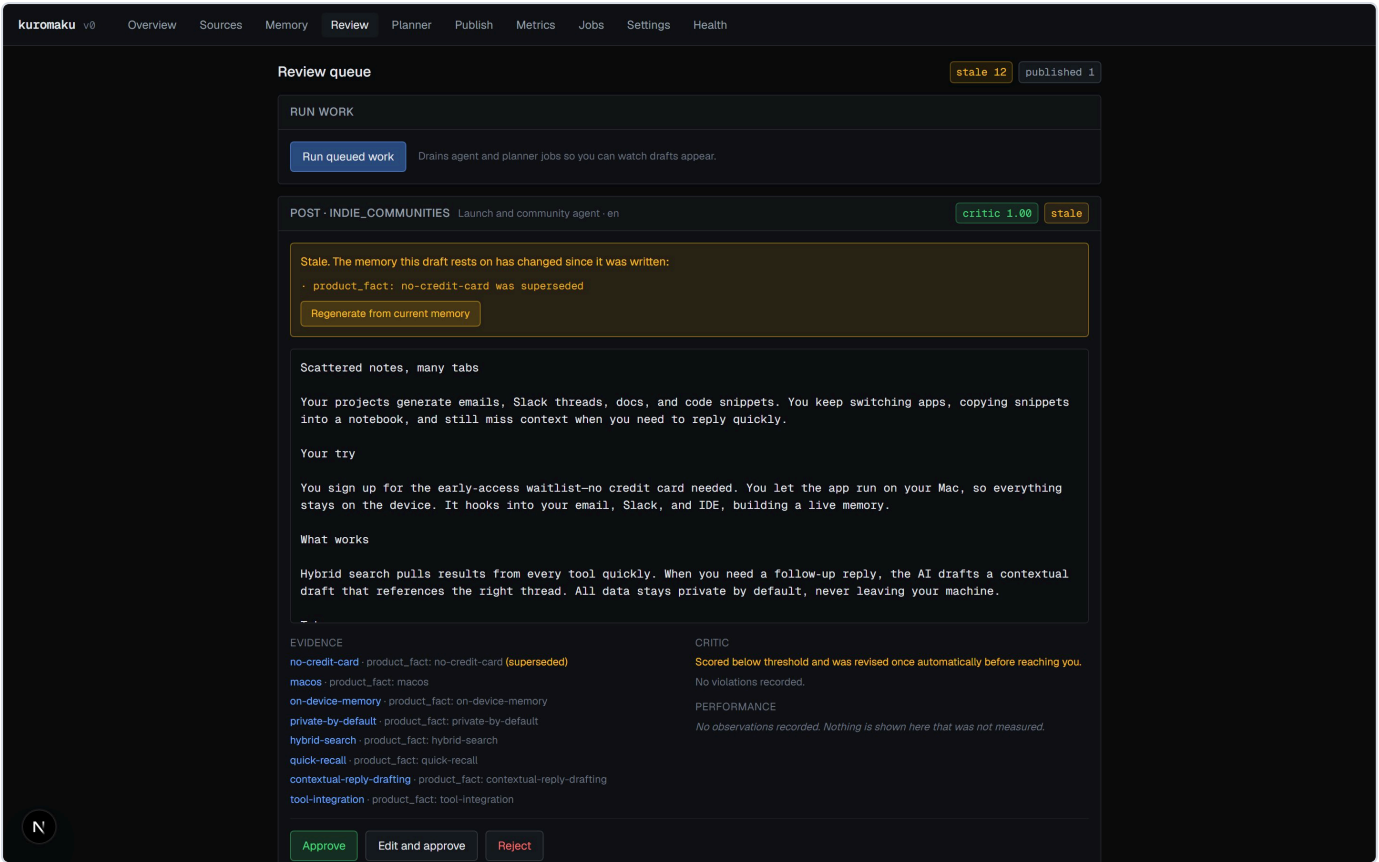
The banner names the record that changed, not just "this is out of date". This draft cited that record directly, so the line is short; when the change is inherited the same banner says how many hops upstream it happened and which record on the path carried it. "Regenerate from current memory" enqueues a fresh run of the same agent.

Nothing polled and nothing was scheduled. Staleness is computed at edit time by walking a derivation graph the compiler wrote:

```
with recursive downstream(id, depth, path) as (  
  select $1::uuid, 0, array[$1::uuid]  
  union all  
  select rd.derived_record_id, d.depth + 1, d.path || rd.derived_record_id  
  from record_derivations rd  
  join downstream d on rd.source_record_id = d.id  
  where d.depth < 16  
         and not rd.derived_record_id = any(d.path)  
)  
select distinct id from downstream
```

That graph is real: 928 edges on the current workspace, with chains four hops deep. The edit captured above reached 39 records and marked 8 artifacts stale. Editing the fact marks every artifact citing any of them.

Propagating one hop would have been much easier and would have looked identical in a demo where the draft happens to cite the edited record directly. It would also have reproduced the original defect one level down: correct a product fact, and the positioning built on it keeps looking current.



The status counts at the top of the queue have changed. Stale is a first class artifact status rather than a warning bolted on, which is what lets the queue be filtered and counted by it.

The defect this answers. Editing the ICP in Okara left every derived draft unchanged and unflagged. There was no way to tell which work rested on a fact you had just corrected. Here the edit names its own consequences, and the version you corrected stays readable next to the one that replaced it.

Step 3. The agent set is derived from the strategy

Go to /planner . The left of this screen is what the strategy compiled. The right is what can actually execute it.

kuzomaku v0

OverviewSourcesMemoryReviewPlannerPublishMetricsJobsSettingsHealth

Planner

2 open coverage gaps2 registered agents

RUN PLANNER

Run planner nowAlso runs on the 5-minute cron.

COVERAGE GAPS

a prioritised channel with no agent — surfaced, not ignored

reddit

rank 3

open

acknowledge

Ranked 3 in the strategy — MacOS-centric and startup subreddits (e.g., r/mac, r/startups) attract founders and privacy-concerned macOS users who need local-first memory, but no registered agent serves this channel, so nothing can be drafted for it.

linkedin

rank 5

open

acknowledge

Ranked 5 in the strategy — Founders use LinkedIn for professional networking and discovering enterprise-grade AI assistants, though outreach requires higher effort, but no registered agent serves this channel, so nothing can be drafted for it.

CHANNEL PRIORITIES VS AGENT COVERAGE

strategy on the left, what can actually execute it on the right

Rank	Channel	Agent	Rationale
1	product_hunt	covered	Founders and early-adopter builders often browse Product Hunt for new AI productivity tools, matching ShogunAI's target of founders and fast-movers.
2	hacker_news	covered	Fast-movers and developer-focused founders discuss cutting-edge AI utilities on Hacker News, making it a good fit for hybrid-search memory tools.
3	reddit	no agent	MacOS-centric and startup subreddits (e.g., r/mac, r/startups) attract founders and privacy-concerned macOS users who need local-first memory.
4	indie_communities	covered	Indie maker forums (Indie Hackers, Maker communities) host fast-movers seeking integrated workflow tools like ShogunAI.
5	linkedin	no agent	Founders use LinkedIn for professional networking and discovering enterprise-grade AI assistants, though outreach requires higher effort.

REGISTERED AGENTS

seeded in code, not the database

The coverage column reads `covered` or `no agent`, computed from the registry at request time. Registering an agent changes it immediately, with no migration and no re-compile.

The registry is seeded in code and is deliberately shorter than the channel list the compiler can produce:

```
/**
 * This list is deliberately shorter than the channel list the compiler can
 * produce. That mismatch is the point: a prioritised channel with no agent here
 * becomes a visible coverage gap instead of silently doing nothing.
 */
```

Two agents cover six channels. The compiler can prioritise ten. What happens to the other four is the entire point of this screen.

COVERAGE GAPS

a prioritised channel with no agent — surfaced, not ignored

reddit

rank 3

open

acknowledge

Ranked 3 in the strategy — MacOS-centric and startup subreddits (e.g., r/mac, r/startups) attract founders and privacy-concerned macOS users who need local-first memory, but no registered agent serves this channel, so nothing can be drafted for it.

linkedin

rank 5

open

acknowledge

Ranked 5 in the strategy — Founders use LinkedIn for professional networking and discovering enterprise-grade AI assistants, though outreach requires higher effort, but no registered agent serves this channel, so nothing can be drafted for it.

Each gap carries the rank the strategy assigned and a sentence saying nothing can draft for it. The planner neither attempted these channels nor skipped them quietly; it wrote a row someone has to look at.

In code the gap is the output, and the `continue` is what makes it so:

```
if (agents.length === 0) {
  await db.insert(coverageGaps).values({ ... })
    .onConflictDoUpdate({ ... });
  result.gaps.push({ channel: entry.channel, rank: entry.rank, rationale });
  continue; // no job is created for this channel
}
```

Where an agent does exist, the job carries the reason it was scheduled.

SCHEDULED WORK every job carries the reason it was scheduled		
queued	indie_communities	inspect
Roadmap item "Share Demo in Indie Maker Communities": Post a short demo video and early-access invitation in Indie Hackers and other maker forums.		
queued	hacker_news	inspect
Roadmap item "Announce ShogunAI on Hacker News": Publish a launch thread on Hacker News linking to the early-access waitlist and demo.		
queued	product_hunt	inspect
Roadmap item "Launch ShogunAI on Product Hunt": Submit ShogunAI with a launch post, demo video, and early-access link to Product Hunt.		
queued	hacker_news	inspect
Channel ranked 2 in the compiled strategy (Fast-movers and developer-focused founders discuss cutting-edge AI utilities on Hacker News, making it a good fit for hybrid-search memory tools.). Launch and community agent covers it, and 1 of 6 recent artifact(s) here have observations.		
done	indie_communities	inspect
Roadmap item "Share Demo in Indie Maker Communities": Post a short demo video and early-access invitation in Indie Hackers and other maker forums.		
done	hacker_news	inspect
Roadmap item "Announce ShogunAI on Hacker News": Publish a launch thread on Hacker News linking to the early-access waitlist and demo.		
done	product_hunt	inspect
Roadmap item "Launch ShogunAI on Product Hunt": Submit ShogunAI with a launch post, demo video, and early-access link to Product Hunt.		
done	hacker_news	inspect
Channel ranked 2 in the compiled strategy (Fast-movers and developer-focused founders discuss cutting-edge AI utilities on Hacker News, making it a good fit for hybrid-search memory tools.). Launch and community agent covers it, and 1 of 4 recent artifact(s) here have observations.		

The reason quotes the compiled rationale verbatim, so the chain from strategy to scheduled work reads without opening the memory browser. It is written to the job row at enqueue time — stored data, not a sentence regenerated for display.

The defect this answers. Okara's own strategy ranked Product Hunt third and founder communities fourth while shipping agents for neither, and shipping UGC video and influencer agents its strategy never asked for. The catalogue was fixed and unrelated to the plan. Here the plan and the catalogue are compared on one screen, and the difference is recorded rather than hidden.

Step 4. Evidence, not justification

Go to `/review` . This is the daily surface.

kuzomaku v0

OverviewSourcesMemoryReviewPlannerPublishMetricsJobsSettingsHealth

Review queue

approved 2draft 3stale 7published 1

RUN WORK

Run queued workDrains agent and planner jobs so you can watch drafts appear.

POST · PRODUCT_HUNT Launch and community agent · en

critic 1.00draft

Your work, remembered.

You get ShogunAI, a macOS-only assistant that captures activity from your daily tools and stores the memory locally. You can draft contextual replies-like follow-up messages-directly in the apps you use. It unifies everything with hybrid search, delivering fast, unified results. Because the memory lives on your device and is private by default, nothing is sent to a cloud server. You can join the early-access waitlist without a credit card.

What's different?

- On-device storage keeps your data private.
- Hybrid search across all captured content, not just files.
- Fast recall and contextual reply drafting built into your workflow.

EVIDENCE

contextual-reply-drafting · product_fact: contextual-reply-drafting

hybrid-search · product_fact: hybrid-search

macos · product_fact: macos

no-credit-card · product_fact: no-credit-card

on-device-memory · product_fact: on-device-memory

private-by-default · product_fact: private-by-default

quick-recall · product_fact: quick-recall

tool-integration · product_fact: tool-integration

CRITIC

Scored below threshold and was revised once automatically before reaching you.

No violations recorded.

PERFORMANCE

No observations recorded. Nothing is shown here that was not measured.

Approve

Edit and approve

Reject

job 056caf32 · created 2026-08-13T22:45:30.280Z

COMMENT_ANGLE · HACKER_NEWS Launch and community agent · en

critic 0.96draft

Fast recall matters when you move quickly. A hybrid search that indexes everything you interact with and returns results quickly cuts the time spent hunting for context. Because the index lives on your Mac and isn't sent to the cloud, you keep data private by default, and you can try it without a credit-card. If you switch between

Channel, kind, agent, critic score and status. Everything needed to triage is on the card.

Beside the content is the evidence panel, and every item in it is a link.

BEFORE

EVIDENCE

no-credit-card · product_fact: no-credit-card (superseded)

macos · product_fact: macos

on-device-memory · product_fact: on-device-memory

private-by-default · product_fact: private-by-default

hybrid-search · product_fact: hybrid-search

quick-recall · product_fact: quick-recall

contextual-reply-drafting · product_fact: contextual-reply-drafting

tool-integration · product_fact: tool-integration

Each entry names the record type and key, and the trailing note explains what the item is. An entry attributed by the runner rather than named by the model says so.

AFTER

kuzomaku v0

OverviewSourcesMemoryReviewPlannerPublishMetricsJobsSettingsHealth

← memory no-credit-cardProduct factsen

VERSION HISTORY append only — editing supersedes, it never overwrites

v10 · activehumanconfidence 0.952026-08-13T22:57:29.544Z

{
 "fact": "No credit card is required to join the early-access waitlist.",
 "note": "Corrected during review.",
 "category": "pricing",
 "correctedByHuman": true
}

Asserted by a human via the memory editor.

v9 · supersededcompiledconfidence 0.902026-08-13T21:07:56.174Z

{
 "fact": "No credit card is required to join the early-access waitlist.",
 "category": "pricing"
}

ShogunAI — Memory that acts

v8 · supersededcompiledconfidence 0.952026-08-13T20:53:37.700Z

{
 "fact": "No credit card is required to get early access.",
 "category": "pricing"
}

ShogunAI — Memory that acts

v7 · supersededcompiledconfidence 0.902026-08-13T20:37:48.656Z

{
 "fact": "No credit card is required to join the early-access waitlist.",
 "category": "pricing"
}

ShogunAI — Memory that acts

v6 · supersededcompiledconfidence 0.902026-08-13T18:05:37.887Z

{
 "fact": "No credit card is required for ShogunAI",
 "category": "pricing"
}

ShogunAI — Memory that acts

The record you land on carries its own sources, confidence and version history. The trail continues: draft, to the record it used, to the page that record came from.

This is what "evidence, not justification" means in practice. There is no prose panel explaining why the draft is good. There is a list of things it rests on, each of which can be opened, checked and corrected.

The evidence requirement is enforced, not encouraged. The runner throws before persisting an artifact that has none:

```
if (draft.evidence.length === 0) {
  throw new Error(
    `Agent "${agentId}" returned an artifact with no evidence. Every draft must carry the memory records
    and links it rests on.`
  );
}
```

Beside the evidence sits the critic, which reviewed the draft before any human saw it.

POST · PRODUCT_HUNT Launch and community agent · en critic 1.00 draft

Your work, remembered.

You get ShogunAI, a macOS-only assistant that captures activity from your daily tools and stores the memory locally. You can draft contextual replies—like follow-up messages—directly in the apps you use. It unifies everything with hybrid search, delivering fast, unified results. Because the memory lives on your device and is private by default, nothing is sent to a cloud server. You can join the early-access waitlist without a credit card.

What's different?

- On-device storage keeps your data private.
- Hybrid search across all captured content, not just files.
- Fast recall and contextual reply drafting built into your workflow.

EVIDENCE

contextual-reply-drafting · product_fact: contextual-reply-drafting

hybrid-search · product_fact: hybrid-search

macos · product_fact: macos

no-credit-card · product_fact: no-credit-card

on-device-memory · product_fact: on-device-memory

private-by-default · product_fact: private-by-default

quick-recall · product_fact: quick-recall

tool-integration · product_fact: tool-integration

CRITIC

Scored below threshold and was revised once automatically before reaching you.

No violations recorded.

PERFORMANCE

No observations recorded. Nothing is shown here that was not measured.

Approve

Edit and approve

Reject

job 056caf32 · created 2026-08-13T22:45:30.280Z

The violation names the voice rule it breaks, quotes the offending phrase, and carries a severity. A draft scoring below 0.7 is revised once and re-scored, so the number shown always describes the text on screen, and the panel says when that happened.

Cost is not hidden either. Every model call is logged with its tokens and price.

compiler:product_facts openai/gpt-oss-120b compile 21:07:52

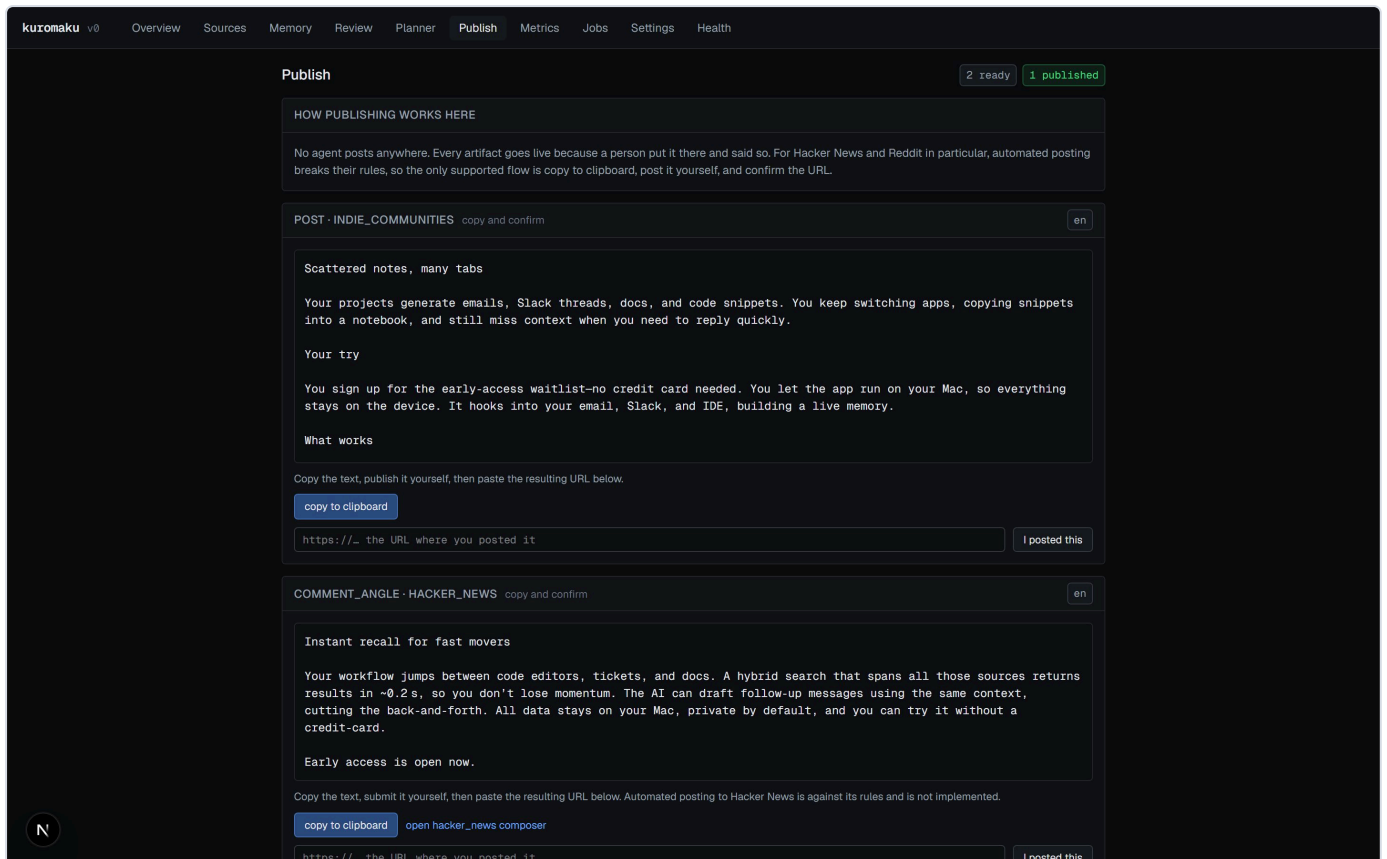
INPUT	OUTPUT	COST	DURATION
2,443	1,382	\$0.001403	3750ms
▶ prompt (9,832 chars)			
▶ raw output (2,469 chars)			

Four measured figures, with the prompt and raw output both expandable. An unpriced model renders as "unpriced" rather than \$0.00, because a zero would claim the call was free rather than that its model has no price entry.

The defect this answers. Okara's drafts carried a prose "why this works" panel with no links, no named threads, no data, and no reference to which memory record informed the choice. Worse, it rendered invented like and view counts on drafts that had never been published. Here every claim about provenance is a link, and every number is either measured or absent.

Step 5. Performance closes the loop

Go to `/publish`.



No agent posts anywhere. For Hacker News and Reddit the instruction says plainly that automated posting breaks their rules and is not implemented. The only supported flow is copy the text, post it yourself, then paste the resulting URL.

That confirmation is gated in code:

```
if (artifact.status !== "approved") {
  throw new Error(
    `Only an approved artifact can be marked as posted. This one is ${artifact.status}.`,
  );
}
```

A draft cannot skip review, and publishing requires a real URL so that performance can later be attributed to it. Once something is live, record what actually happened. Here is the performance panel before and after a single observation.

THE PERFORMANCE PANEL WITH NOTHING MEASURED, AND AFTER ONE OBSERVATION

BEFORE

OBSERVATIONS

recorded, never generated — nothing here was projected

No performance has been observed yet.

This is deliberately blank rather than showing zeros. A zero would claim something was measured and found to be nothing; an empty state says nothing was measured. Record an observation from the [publish](#) screen once something is live.

No zeros. It says no performance has been observed yet, and explains why that differs: a zero claims something was measured and found to be nothing.

AFTER

OBSERVATIONS

recorded, never generated — nothing here was projected

upvotes: 42 across 1

Metric	Value	Channel	Source	Observed
upvotes	42	hacker_news	manual	2026-08-13 22:58

The figure, its metric, its channel and its source. Observations are only ever inserted; no code path derives, estimates or backfills one.

Now the part that makes this a loop rather than a report. Here is the planner scheduling a channel, and the planner declining to schedule that same channel on a later run.

THE PLANNER SCHEDULING A CHANNEL, THEN GATING IT

BEFORE

SCHEDULED WORK every job carries the reason it was scheduled

queued indie_communities inspect
Roadmap item "Share Demo in Indie Maker Communities": Post a short demo video and early-access invitation in Indie Hackers and other maker forums.
queued hacker_news inspect
Roadmap item "Announce ShogunAI on Hacker News": Publish a launch thread on Hacker News linking to the early-access waitlist and demo.
queued product_hunt inspect
Roadmap item "Launch ShogunAI on Product Hunt": Submit ShogunAI with a launch post, demo video, and early-access link to Product Hunt.
queued hacker_news inspect
Channel ranked 2 in the compiled strategy (Fast-movers and developer-focused founders discuss cutting-edge AI utilities on Hacker News, making it a good fit for hybrid-search memory tools.). Launch and community agent covers it, and 1 of 6 recent artifact(s) here have observations.

The channel is ranked, covered by an agent, and nothing blocks it.

AFTER

RUN PLANNER

Planning...

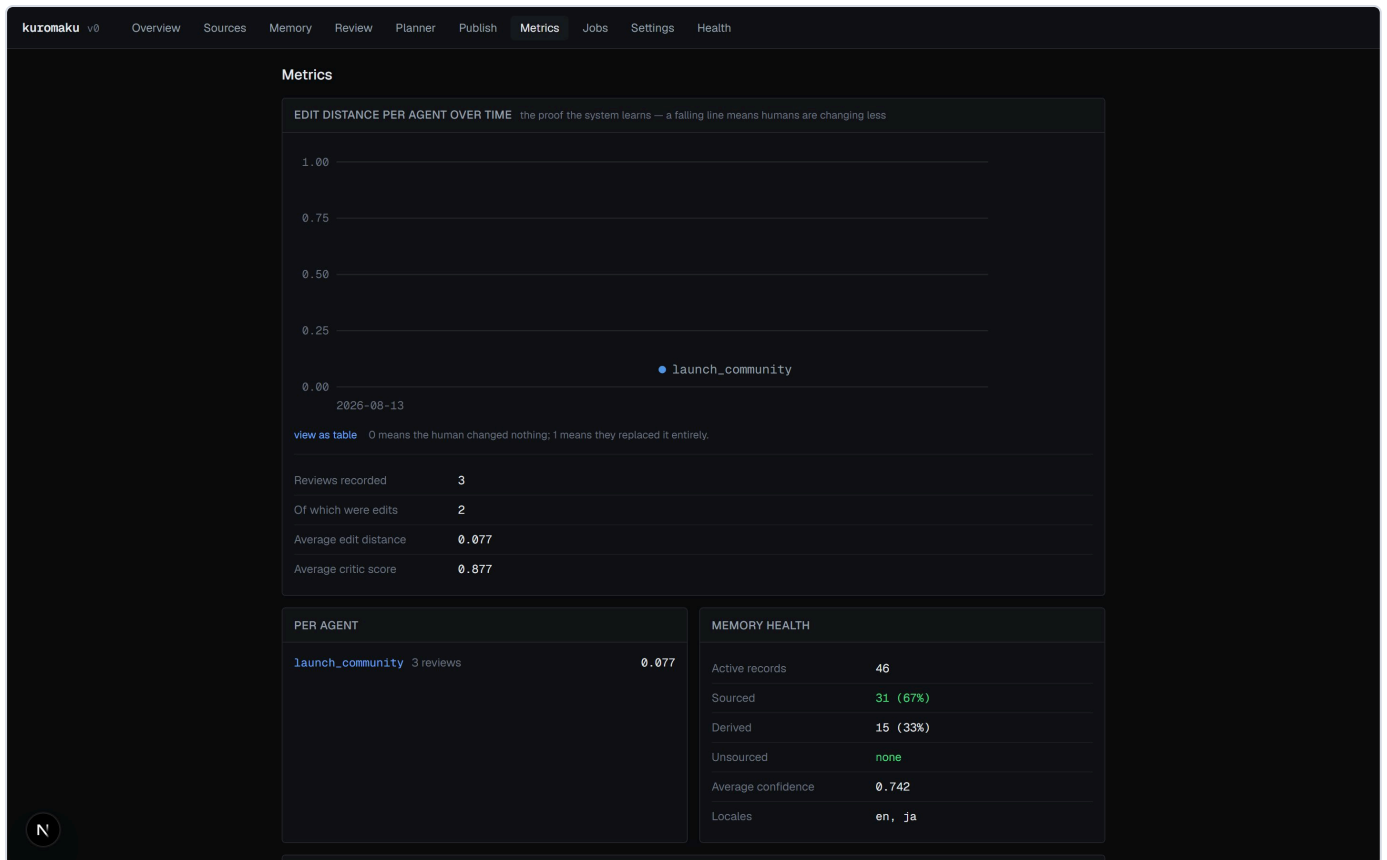
 Also runs on the 5-minute cron.

Recent artifacts exist in that channel and none carry an observation, so the planner declines to draft more until someone measures the work already done.

The rule is two conditions, both required:

```
if (recentStats && recentStats.total >= 2 && recentStats.observed === 0) {
  const why =
    `${recentStats.total} artifact(s) in the last ${RECENT_WINDOW_DAYS} days and no observations
    recorded ` +
    `for any of them. Record performance for this channel before drafting more.`;
  result.skipped.push({ channel: entry.channel, why });
  continue;
}
```

A single recent draft is not enough to gate: one draft is not a pattern. And a channel with any observation is not gated regardless of what the number says. The rule is about whether anyone is measuring, not whether the results were good.



The edit distance chart draws only the days that actually have reviews. No zero fill, no interpolation, which is why a single day of reviews shows points and no line. The chart is correct; the claim it exists to support needs weeks of data this system has not accumulated, and section 6 says so.

The defect this answers. Okara's entire pipeline completed with Search Console and Analytics skipped, and nothing degraded. Performance data was never an input to planning. Here the absence of measurement is itself an input, and it changes what gets scheduled.

5. How it is verified

There is no unit test suite. There are three executable scripts that assert against a real database, two of them against real model calls. Section 6 states what that costs.

`npm run verify` runs the infrastructure checks with no model calls: environment, connectivity, migration state, the encryption round trip, and the whole queue contract. It reads the raw key column back to confirm the plaintext does not appear in it, and it asserts that two concurrent claims take different rows, a throwing job requeues with backoff, exhausted attempts mark it failed and release its key, and a job whose worker died is recovered rather than stranded. It is non destructive: any existing key is restored and every job it creates is deleted.

`npm run e2e` walks the definition of done against real model calls. The last full run passed all 21 checks. Selected evidence from that run:

- Re-compiling supersedes rather than duplicating: 46 active against 216 superseded, 203 rows carrying a predecessor id, versions reaching 10
- Every record accounts for itself: 31 sourced, 15 derived, 0 ungrounded
- No derived record more confident than what it rests on, across all 15
- Coverage gaps: Reddit at rank 3, LinkedIn at rank 5

- A draft with 10 evidence items and a critic score of 0.85
- A recorded edit distance of 0.0857
- The planner gating an unobserved channel with the reason quoted above
- Editing a cited record marking 2 artifacts stale and creating one successor
- 116 model calls logged, 112 of them priced, \$0.1164 total

`npm run eval` runs a golden set of twenty fixed cases against real model calls. It departs from the brief it was written against, and says so in the file:

```
/**
 * A deliberate departure from the spec's wording, flagged rather than done
 * silently: the spec asks for "20 fixed prompts with reference outputs", but
 * exact-match reference outputs are meaningless for open-ended drafting.
 */
```

Exact match references would measure temperature, not quality. Each case instead pairs a fixed input with assertions over the output: JSON validity, every record addressable, confidence in range, no uncited record claiming 0.5 or above, no fabricated metrics, no invented thread URLs, and one negative case where a draft stuffed with "trusted by 50,000 developers" and "10x faster" must score below the critic threshold.

Bugs these checks actually caught

Verification is only worth describing if it found something.

Query normalisation. Two spellings of one search query produced two different cache keys and called the provider twice, which is precisely the duplicate execution defect from section 2. The cause was ordering: terminal punctuation was stripped before trimming, so a trailing space kept the anchor from ever matching. It survived until a search API key made the integration check runnable. The fix is three characters of reordering, and the invariant is now a permanent check.

Stage truncation. A compile stage emitted enough records to hit the output token cap, leaving incomplete JSON. The retry then made it worse: told only that the JSON was invalid, the model returned the same too long answer. The fix distinguishes the two cases and tells the retry it has a length problem.

Fixtures leaking into real state. The pair capture script inserted two drafts directly, bypassing the runner that enforces the evidence invariant, and never removed them. The interface flagged them correctly, with "No evidence. This should be impossible", and that warning reached a screenshot in a draft of this document. The script now attaches evidence like any real draft and deletes its fixtures once the pair is captured. The same review found the observation fixture accumulating one row per run, under a caption reading "a single observation".

A runaway recursive query. Introduced while building the derivation graph for this document. `UNION` deduplicates the whole row, so a node reachable at several depths produced one row per depth and diamond shaped paths multiplied out. It hung a capture run on a graph of 199 edges. Carrying the visited path and excluding nodes already on it fixed it; the graph has since grown to 928 edges and the same query returns in under a second.

An aggregate that quietly returned zero. The memory health header counts sourced, derived and ungrounded records in one pass. Expressed through the query builder, the correlated subqueries failed to bind the outer row, so every bucket came back empty: a memory with 31 sourced records reported none, and the page said so in its header without complaining. Nothing threw. The fix is the same statement written raw with its own

alias, and the embarrassing part is that the number was on screen for a while before I read it rather than glanced at it.

A test suite grading the wrong papers. `npm run verify` enqueues its own jobs and asserts on what becomes of them, but the worker claims the oldest queued row of any type. With real work sitting in the queue it was claiming that instead, and seven checks failed describing rows the script had never created. `claimNext` now takes an optional type filter and the script uses it. The queue was fine the whole time.

A golden set testing a layer nothing uses. Six compiler cases went red the moment the compile model changed, reporting records with no key and no confidence — while the compiler itself was producing perfectly good keyed records from the same output. The assertions read the raw JSON; the compiler never does, because `emittedRecord` normalises first and derives a key from the payload when the model omits one. The assertions now run against the normalised record, which is the thing the system actually stores.

An unbounded sleep inside a job. On a long `retry-after`, a rate-limited call sat waiting while holding its lock. One drain spent twenty-three minutes with no model call in flight and no way to tell from outside whether it was working or wedged. A job now refuses to wait longer than ninety seconds and hands itself back to the queue, which already knows how to retry with backoff. Scripts, which have no queue to fall back on, raise the ceiling explicitly.

6. Limits and what comes next

Not implemented

Google Search Console. An optional intake path in the brief. No OAuth flow, no client, no sync job. The `observation_source` enum contains `gsc` and nothing writes it.

GitHub pull requests for blog content. Markdown export exists. Repository integration does not.

The three pane layout. The brief asks for memory left, work centre, agents and jobs right. The interface is one pane per route with a top nav. The information is all present; the layout is not what was asked for.

Per locale agent runs. Voice rules compile per locale and Japanese rules exist in the memory, but the planner hard codes English on every job, so those rules are compiled and never used.

The `classify task` and `settings.model_config`. Both exist because the brief names them. Neither is read by any code path.

Shortcuts, and what they cost

There are no transactions. The Neon HTTP driver runs every statement in its own implicit transaction, so writing a new record version and superseding the old one are two separate operations with a window between them. A crash in that window leaves two active versions. Compiles are serialised by the queue so it is not reachable in normal use, but it is real.

One active record per key is enforced in application code, not the database. The partial unique index that would enforce it cannot be added while the insert happens before the supersede. Fixing it properly means the transaction work above.

Derivation edges over approximate. Every record a stage emits gets an edge to every record in that stage's dependency slice, because the stage saw them together and cannot report which it actually used. This marks slightly more stale than strictly necessary, which is the safer direction, but the count is an upper bound.

A re-compile supersedes records without marking derived artifacts stale. Only human edits propagate. This was found while capturing screenshots for this document: after three re-compiles, no existing draft cited a record that was still active. It is arguably the more common path in production and should propagate too.

The critic's original draft is discarded. When the critic revises a low scoring draft, the pre-revision text is not stored, so the critic's own effect cannot be measured. Only the human's, through edit distance.

No pagination anywhere, in the interface or the API. Lists are capped and a workspace exceeding those caps silently sees a truncated view.

No authentication at all. Every control described in this document assumes a trusted operator. The public demo instance runs with `DEMO_MODE=1`, which refuses the four actions that spend money or touch credentials — compiling, crawling, draining the queue, and writing a model key. Everything else, including editing memory and approving drafts, is open to anyone who finds the URL, because being checkable is the point of that instance. This is a refusal list, not authentication: it protects the owner's wallet and key, not the data, and any deployment holding something worth protecting needs the real thing.

No unit tests, no UI tests, no CI, nothing hermetic. The three scripts all require a live database, and two require a live model provider. The cost was concrete and is described in section 5.

What I would build next, in order

Move to the WebSocket driver and get transactions. This unblocks the active record unique index, makes editing atomic, and lets the claim query return to its conventional shape. Most of the correctness gaps above disappear with it, so it comes first.

Propagate staleness on re-compile, not only on human edit. The machinery already exists. The compiler simply does not call it.

Make the compiler resumable. Stages already commit independently. Recording which stages completed against which source version would turn a ten minute all or nothing job into restartable work, which fixes the serverless timeout problem properly rather than working around it.

Persist the compile summary. Per stage counts exist only as text in a job log. A table would make memory quality a trend rather than an anecdote, and the sourced-to-derived ratio over time is the best single indicator of whether the crawl and the prompts are improving.

Unit tests for the pure functions. Edit distance, duration parsing, slug generation, URL canonicalisation, query normalisation, the envelope normaliser. All pure, all edge case heavy, all untested, and demonstrably where the bugs have been.

Then authentication, then Search Console, then the two agents the system itself identified. Reddit and LinkedIn are the channels the compiler ranked and the registry cannot serve. Writing them is roughly a day each given the agent contract, and it would demonstrate the extension path on gaps the system found rather than ones chosen in advance.